

C++ 中常用的 STL

VECTOR 基本用法

STL是什么

- STL 是 **Standard Template Library** 的英文缩写，意思是：标准模板库。
- STL 的代码从广义上讲分为三类：**algorithm**（算法）、**container**（容器）和 **iterator**（迭代器）
- 几乎所有的代码都采用了模板类和模版函数的方式，这些现成的算法、数据结构模板，能有效地提高代码效率和正确性。
- 参考资料：<http://zh.cppreference.com/>

容器 (container)

- 经典的数据结构数量有限，但是我们常常重复着一些为了实现向量、链表等结构而编写的代码，这些代码都十分相似，只是为了适应不同数据的变化而在细节上有所出入。
- **STL**容器对最常用的数据结构提供了支持，这些模板的参数允许我们指定容器中元素的数据类型，可以将我们许多重复而乏味的工作简化。
- 容器部分主要由
`<vector>`, `<list>`, `<deque>`, `<set>`, `<map>`, `<stack>`
和 `<queue>` 组成。

Vector简介

- **Vector**是一种动态数组，是基本数组的类模板。
- **vector** 的存储是自动管理的，按需扩张收缩。
- 需要头文件`<bits/stdc++.h>`或头文件`<vector>`
- 其内部定义了很多基本操作，包括插入、删除、访问等，使用起来十分方便。
- 在开启O2优化的情况下，**Vector**的访问速度甚至能够快过一般的数组，在**STL**的日益普及下，**Vector**必将被广泛应用。

Vector的定义与赋值

- 如图，**Vector**既然是一类数组，那它就能够当做数组定义、使用、赋值。
- **Vector**中可以定义的类型不限，既可以是 **int**，**char**这样的类型，也可以是结构体，甚至是**Vector**。

```
Definition.cpp
1  #include<bits/stdc++.h>
2  using namespace std;
3  int main() {
4      vector<int> a;
5      vector<int> b = {1, 2};
6      cout << b[0] << ' ' << b[1] << endl;
7      int arr[3] = {1, 2, 3};
8      vector<int> c (arr, arr + 3);
9      cout << c[0] << ' ' << c[1] << ' ' << c[2] << endl;
10     vector<char> d = {'A', 'B', 'C'};
11     cout << d[0] << ' ' << d[1] << ' ' << d[2] << endl;
12     return 0;
13 }
```

```
D:\XYX\Programming\Programming\2017-08-2
1 2
1 2 3
A B C

-----
Process exited after 0.03389 seconds
请按任意键继续. . .
```

Vector的Size与Push_back操作

- **Push_back(x)**是**Vector**的成员函数，它能够在**Vector**的末尾加入一个元素**x**。
- **Size()**也是**Vector**的成员函数，其返回值是**Vector**中的元素个数。注意，访问不在**Vector**中的位置是未定义的行为。

Pushback.cpp

```
1  #include<bits/stdc++.h>
2  using namespace std;
3  int main() {
4      vector<int> a = {1, 2};
5      cout << a[0] << ' ' << a[1] << ' ' << a[2] << endl;
6      cout << a.size() << endl;
7      a.push_back(3);
8      cout << a[0] << ' ' << a[1] << ' ' << a[2] << endl;
9      cout << a.size() << endl;
10     return 0;
11 }
```

D:\XYX\Programming\Programming\2017-08

```
1 2 30671192
2
1 2 3
3

-----
Process exited after 0.3109 seconds
请按任意键继续. . .
```

Vector的Begin、End与iterator

- 在每种**STL**容器中都定义了自己的迭代器类型。
- 迭代器(**iterator**)是一种检查容器内元素并遍历元素的数据类型。
- 迭代器相当于一种指针，是容器中一个元素的地址，(*迭代器)才会指向具体的元素。
- 迭代器的++，--运算被重载过，详见下页实例。
- **Begin()**, **End()**是**Vector**的成员函数，返回值分别是**Vector**中**首个元素**的迭代器和**Vector**中**末尾元素向下一位**的迭代器

Vector元素的遍历

- 结合实例，我们可以进一步理解**iterator**的使用方式。
- 下面的循环中**i++**也可以改写为**i+=1**或**i=i+1**，可以理解为将**i**指向下一个位置。

```
Visit.cpp
1  #include<bits/stdc++.h>
2  using namespace std;
3  int main() {
4      vector<int> a = {1, 2, 3};
5      for (unsigned i = 0; i < a.size(); i++)
6          cout << a[i] << ' ';
7      cout << endl;
8      vector<int> :: iterator i;
9      for (i = a.begin(); i != a.end(); i++)
10         cout << *i << ' ';
11     cout << endl;
12     return 0;
13 }
```

```
D:\XYX\Programming\Programming\2017-08-
1 2 3
1 2 3

-----
Process exited after 0.03745 seconds
请按任意键继续. . .
```


vector应用——谁的孙子最多

给定一棵树，其中1号节点是根节点，问哪一个节点的孙子节点最多，有多少个。（孙子节点，就是儿子节点的儿子节点。）

【输入要求】

第一行一个整数 N ($N \leq 100000$)，表示树节点的个数。
此后 N 行，第 i 行包含一个整数 C_i ，表示 i 号节点儿子节点的个数，随后共 C_i 个整数，分别表示一个 i 号节点的儿子节点。

【输出要求】

一行两个整数，表示孙子节点最多的节点，以及其孙子节点的个数，如果有多个，输出编号最小的。

【输入样例】

```
5
2 2 3
1 4
0
1 5
0
```

【输出样例】

```
1 1
```

vector应用——谁的孙子最多II

给定一棵树，其中1号节点是根节点，问哪一个节点的孙子节点最多，有多少个。（孙子节点，就是儿子节点的儿子节点。）

【输入要求】

第一行一个整数 N ($N \leq 100000$)，表示树节点的个数。
此后 $N-1$ 行，第 i 行包含一个整数 F_i ，表示 $i+1$ 号节点的父亲。

【输出要求】

一行两个整数，表示孙子节点最多的节点，以及其孙子节点的个数，如果有多个，输出编号最小的。

【输入样例】

```
5
1
1
2
4
```

【输出样例】

```
1 1
```

Vector的Insert操作

- `Insert(x, y)`是Vector的成员函数，其中，`x`是一个迭代器，`y`是一个具体的值。
- `Insert(x, y)`在`x`对应的元素之前插入了一个值为`y`的元素。
- 由于Vector的分块存储方式，`Insert`的复杂度为 $O(\log(\text{size()}))$ 。

```
Insert.cpp
1  #include<bits/stdc++.h>
2  using namespace std;
3  int main() {
4      vector<int> a = {0, 1, 2};
5      for (unsigned i = 0; i < a.size(); i++)
6          cout << a[i] << ' ';
7      cout << endl;
8      a.insert(a.begin() + 1, 3);
9      for (unsigned i = 0; i < a.size(); i++)
10         cout << a[i] << ' ';
11     cout << endl;
12     return 0;
13 }
```

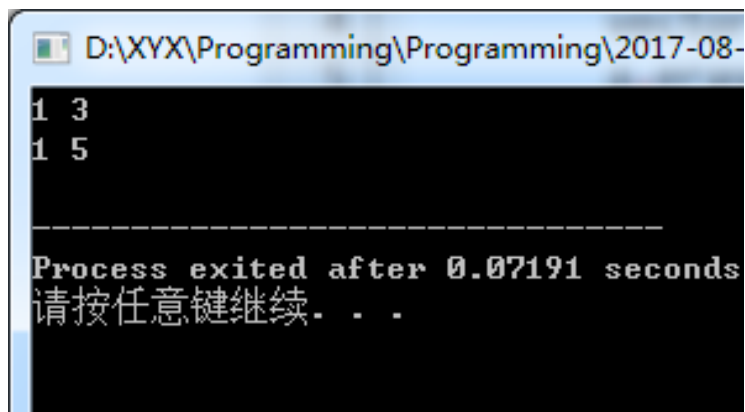
```
D:\XYX\Programming\Programming\2017-08-
0 1 2
0 3 1 2

-----
Process exited after 0.03892 seconds
请按任意键继续. . .
```

Vector的Erase操作

- `Erase(x)`, `Erase(x,y)`是Vector的成员函数，其中，`x,y`是迭代器。
- 分别能够删除`x`处的元素或区间`[x,y)`内的元素。
- 由于Vector的分块存储方式，`Erase`的复杂度为 $O(\log(\text{size()}))$ 。

```
Delete.cpp
1  #include<bits/stdc++.h>
2  using namespace std;
3  int main() {
4      vector<int> a = {1, 2, 3};
5      a.erase(a.begin() + 1);
6      for (unsigned i = 0; i < a.size(); i++)
7          cout << a[i] << ' ';
8      cout << endl;
9      vector<int> b = {1, 2, 3, 4, 5};
10     b.erase(b.begin() + 1, b.begin() + 4);
11     for (unsigned i = 0; i < b.size(); i++)
12         cout << b[i] << ' ';
13     cout << endl;
14     return 0;
15 }
```



```
D:\XYX\Programming\Programming\2017-08-
1 3
1 5

-----
Process exited after 0.07191 seconds
请按任意键继续. . .
```

Vector的Clear操作

- **Clear()**是Vector的成员函数，使用后**将Vector的Size()设置为0**。
- 然而，我们看到，**Clear**之后，元素并没有被删除，空间也没有释放。
- 因此，**Clear**是 $O(1)$ 的。

```
Clear.cpp
1  #include<bits/stdc++.h>
2  using namespace std;
3  int main() {
4      vector<int> a = {1, 2, 3};
5      for (unsigned i = 0; i < a.size(); i++)
6          cout << a[i] << ' ';
7      cout << endl;
8      a.clear();
9      for (unsigned i = 0; i < a.size(); i++)
10         cout << a[i] << ' ';
11     cout << endl;
12     for (unsigned i = 0; i < 3; i++)
13         cout << a[i] << ' ';
14     cout << endl;
15     return 0;
16 }
```

```
D:\XYX\Programming\Programming\2017-08
1 2 3
1 2 3
-----
Process exited after 0.1398 seconds
请按任意键继续. . .
```

vector应用——链表操作

给定一个**N**个数的数组，**M**次操作，每次操作为下列操作之一。求最后的数组。

操作**1**：在第**X**个数之后插入一个数**Y**。

操作**2**：删除第**X**个数。

【输入要求】

第一行两个整数**N**，**M** (**N**，**M**≤100000) 含义见试题描述。

第二行**N**个整数，表示原来的数组。

接下来**M**行，每行第一个数**OPT**，表示操作类型。

对于操作**1**，接下来两个数**X**，**Y**，含义见题面描述，保证 $0 \leq X \leq$ 当前数的个数，若**X=0**，表示在数组开头插入。

对于操作**2**，接下来一个数**X**，含义见题面描述，保证 $1 \leq X \leq$ 当前数的个数。

【输出要求】

输出若干个数，表示最后的数组。

【输入样例】

5 3

1 2 3 4 5

1 1 6

2 1

2 2

【输出样例】

6 3 4 5

Algorithm库函数在Vector的应用

- **Sort(x, y)**对于区间[x,y)实现了排序。同样，它也可以用于Vector。
- 类似地，**Reverse(x, y)**对区间[x,y)实现了翻转。它同样能够作用在Vector中。

```
Algorithms.cpp
1  #include<bits/stdc++.h>
2  using namespace std;
3  int main() {
4      vector<int> a = {5, 2, 3, 4, 1};
5      sort(a.begin(), a.end());
6      for (unsigned i = 0; i < a.size(); i++)
7          cout << a[i] << ' ';
8      cout << endl;
9      reverse(a.begin() + 1, a.begin() + 4);
10     for (unsigned i = 0; i < a.size(); i++)
11         cout << a[i] << ' ';
12     cout << endl;
13     return 0;
14 }
```

```
D:\XYX\Programming\Programming\2017-08-
1 2 3 4 5
1 4 3 2 5

-----
Process exited after 0.03736 seconds
请按任意键继续. . .
```

vector之sort应用——数组操作

今有**N**个数组，初始时，**N**个数组均为空。共有**M**次操作，每次在第**X**个数组中加入数字**Y**。问最终各数组中有多少数，并将它们排序输出。

【输入要求】

第一行两个整数**N**、**M** ($N \leq 100000$, $M \leq 300000$)。

接下来**M**行，每行两个整数**X**、**Y**，含义见试题描述。

($1 \leq X \leq N$, $Y \leq 10^9$)

【输出要求】

共**N**行，第**i**行第一个数**SUM**，表示第**i**个数组数的个数，接下来**SUM**个数，为排序之后的数组。

【输入样例】

3 5

1 3

1 2

1 1

2 1

3 1

【输出样例】

3 1 2 3

1 1

1 1

vector之sort应用——排序

给定**N**个数组，要求先对这**N**个数组分别进行排序，然后再根据**N**的数组的字典序对这**N**个数组进行排序。输出排序的结果。

【输入要求】

第一行一个整数**N**，表示数组数。

接下来**N** ($N \leq 1000$) 行，每一行先包含一个整数**SUM** ($SUM \leq 1000$)，表示数组的大小，接下来**SUM**个整数，表示数组中的一个元素。

【输出要求】

共**N**行，每行表示一个数组。

【输入样例】

```
4
1 3
1 1
2 2 1
3 2 3 1
```

【输出样例】

```
1
1 2
1 2 3
3
```

vector之reverse应用——序列翻转

给定一个**N**个数的数组，**M**次操作。每次操作将数组的一段翻转，求最后的数组。

【输入要求】

第一行两个整数**N**，**M** (**N**，**M**≤1000) 含义见试题描述。

第二行**N**个整数，表示原来的数组。

接下来**M**行，每行两个整数**X**，**Y** ($1 \leq X \leq Y \leq N$)，表示翻转区间**[X,Y]**。

【输出要求】

一行**N**个整数，表示操作后的数组。

【输入样例】

5 2

1 2 3 4 5

2 4

4 5

【输出样例】

1 4 3 5 2

在Vector中删除某关键字的元素

- **Remove** 移动指定区间中的元素直到所有元素在“不删除的”元素在区间的位置和原来它们的一个指向最后一个“不删除的”元素的迭代器。
- 所以，我们用前面讲到**Erase**即可删除某关键字的元素

```
Delx.cpp
2 using namespace std;
3 vector<int> a;
4 int main() {
5     int n, x;
6     cin >> n;
7     for (int i = 1; i <= n; i++)
8         a.push_back(rand() % 11);
9     for (unsigned i = 0; i < a.size(); i++)
10         cout << a[i] << ' ';
11     cout << endl;
12     cin >> x;
13     a.erase(remove(a.begin(), a.end(), x), a.end());
14     for (unsigned i = 0; i < a.size(); i++)
15         cout << a[i] << " ";
16     cout << endl;
17     return 0;
18 }
```

```
D:\XYX\Programming\Programming\2017-0
10
8 9 9 1 7 5 5 10 1 0
5
8 9 9 1 7 10 1 0

-----
Process exited after 3.268 seconds
请按任意键继续. . .
```

Vector的比较

- **Vector**重载了比较运算符，比较的结果是字典序比较的结果。
- 所谓字典序比较，就是类似于字符串的比较，按位比较，有结果则结束。

```
Compare.cpp
1  #include<bits/stdc++.h>
2  using namespace std;
3  int main() {
4      vector<int> a = {1, 2};
5      vector<int> b = {1, 2, 3};
6      vector<int> c = {1, 3, 2};
7      if (a > b) puts("A is greater than B.");
8      else puts("B is greater than A.");
9      if (a > c) puts("A is greater than C.");
10     else puts("C is greater than A.");
11     if (b > c) puts("B is greater than C.");
12     else puts("C is greater than B.");
13     return 0;
14 }
```

```
D:\XYX\Programming\Programming\2017-08-2
B is greater than A.
C is greater than A.
C is greater than B.

-----
Process exited after 0.03028 seconds
请按任意键继续. . .
```

SET的基本用法

SET是什么

- **set** 是 STL 中一种标准关联容器，封装了一种高效的平衡检索二叉树——红黑树。
- **set** 是用来存储同一数据类型的集合，在 **set** 中每个元素的值都唯一，而且系统能根据元素的值自动进行排序。
- 支持 $O(\log(\text{size}))$ 复杂度的插入、删除、查找操作。虽然存在一定的常数，但开启 **O2** 优化后效率非常高。
- **set** 不支持插入重复的元素。若要插入重复元素可使用
- **multiset**，其用法与 **set** 几乎完全相同。
- **set** 中数元素的值不能直接被改变。

SET的构造

- 直接用类似 STL 其他容器的方式构造Set。Set中的元素可以是任意类型的，但是由于需要排序，所以元素必须有一个序，即大小的比较关系。

[*] 未命名1.cpp

```
1  #include<bits/stdc++.h>
2  using namespace std;
3  set<int> a; //元素类型为int的set
4  set<char> b; //元素类型为char的set
5  set<string> c; //元素类型为string的set
6  int main()
7  {
8      return 0;
9  }
```

SET的赋值

- 如图，c++11中，set可以像数组一样赋初值，但是赋值完成后set中的元素是自动排好序的。
- set没有尾部插入函数push_back()，元素的插入一般使用insert进行动态检索插入。
- **a.insert(x)** --在集合中a中插入元素x，x的类型必须与set的元素类型一致。如果插入的元素在set中已存在则会忽略。

```
未命名1.cpp  输出.cpp
1  #include<bits/stdc++.h>
2  using namespace std;
3  set<int> a={2,1};
4  int main()
5  {
6      a.insert(5);
7      a.insert(2);
8      set<int>::iterator it;
9      for(it=a.begin();it!=a.end();it++)
10         cout<<*it<<" ";
11     cout<<endl;
12     return 0;
13 }
```

F:\c++\备课\STL\set\未命名1.exe

```
1 2 5

-----
Process exited after 0.4243 seco
请按任意键继续. . .
```


SET的BEGIN、END与ITERATOR

- 迭代器(iterator)是一种检查容器内元素并遍历元素的数据类型。
- set的迭代器是封装了元素节点的指针, (*迭代器)才会指向具体的元素。
- set 的迭代器仅支持 ++ 和 --, 不支持 += 和 -=。这意味着无法快速定位到 set 中的第 k 个元素。
- Begin(), End()是set的成员函数, 返回值分别是set中首个元素的迭代器和set中末尾元素向后的迭代器。

SET的输出

- `set` 不能像数组的输出那样使用下标输出，需要使用迭代器依次遍历。
- 使用迭代器时，要写成 `it!=a.end()`;
- 输出的是 `*it`。

```
[*] 未命名1.cpp
1  #include<bits/stdc++.h>
2  using namespace std;
3  set<int> a={2,1};
4  int main()
5  {
6      a.insert(5);
7      set<int>::iterator it;
8      for(it=a.begin();it!=a.end();it++)
9          cout<<*it<<" ";
10     cout<<endl;
11     return 0;
12 }
```

```
F:\c++\备课\STL\set\未命名1.exe
1 2 5
-----
Process exited after 0.4243 seco
请按任意键继续. . .
```

SET的BEGIN() 和RBEGIN()

- **set** 中的元素总是保持单调递增。因此：
- **begin()** 返回的迭代器指向 **set** 中的最小值；
- **rbegin()** 返回的迭代器指向 **set** 中的最大值。

```
未命名1.cpp  rbegin.cpp
1  #include<bits/stdc++.h>
2  using namespace std;
3  set<int> a={2,1};
4  int main()
5  {
6      a.insert(5);
7      cout<<*a.begin()<<endl;
8      cout<<*a.rbegin()<<endl;
9      return 0;
10 }
```

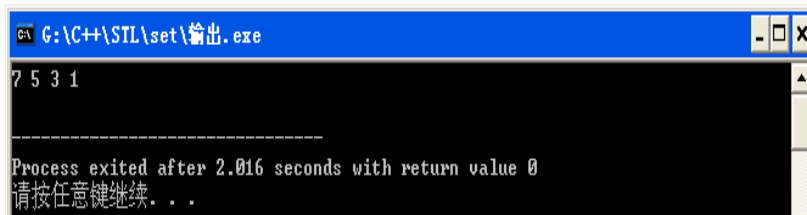
```
F:\c++\课\STL\set\rbegin.exe
1
5
-----
Process exited after 0.5839 s
请按任意键继续. . .
```

SET的END() 和REND()

- **set** 中的元素总是保持单调递增。
- **end()** 返回的迭代器指向 **set** 中的最后元素的后一个位置；
- **rend()** 返回指向集合中第一个元素的前一个位置的迭代器

从大到小输出set中的元素

```
#include<bits/stdc++.h>
using namespace std;
set<int> a={1,3,5,7};
int main( )
{
    set<int>::reverse_iterator rit; //定义反向迭代器
    for(rit=a.rbegin();rit!=a.rend();rit++)
        if(rit==a.rbegin())cout<<*rit;
        else cout<<" "<<*rit;
    cout<<endl;
    return 0;
}
```



```
G:\C++\STL\set\输出.exe
7 5 3 1
-----
Process exited after 2.016 seconds with return value 0
请按任意键继续...
```

SET 中 SIZE、EMPTY 和 CLEAR

- **Size()** 是 **set** 的成员函数，其返回值一个无符号整数，表示 **set** 中元素的个数。时间复杂度 $O(1)$ 。
- **empty()** 返回一个 **bool** 类型，表示 **set** 是否为空。时间复杂度 $O(1)$ 。
- **clear()** 清除 **set** 中的所有元素。时间复杂度 $O(\text{size})$ 。

未命名1.cpp

```
1  #include<bits/stdc++.h>
2  using namespace std;
3  set<int> a={2,1};
4  int main()
5  {
6      a.insert(5);
7      cout<<a.size()<<endl;
8      cout<<a.empty()<<endl;
9      a.clear();
10     set<int>::iterator it;
11     for(it=a.begin();it!=a.end();it++)
12         cout<<*it<<" ";
13     cout<<endl;
14     cout<<a.empty()<<endl;
15     return 0;
16 }
```

```
3
0

1

-----
Process exited after 0.4002
请按任意键继续. . .
```

SET应用——RANDOM

明明想在学校中请一些同学一起做一项问卷调查，为了实验的客观性，他先用计算机生成了N个1到1000之间的随机整数（ $N \leq 100$ ），对于其中重复的数字，只保留一个，把其余相同的数去掉，不同的数对应着不同的学生的学号。然后再把这些数从小到大排序，按照排好的顺序去找同学做调查。请你协助明明完成"去重"与"排序"的工作。

输入要求

第1行为1个正整数，表示所生成的随机数的个数： N （ $N \leq 100$ ）

第2行有N个用空格隔开的正整数，为所产生的随机数。

输出要求

第1行为1个正整数M，表示不相同的随机数的个数。

第2行为M-1个用空格隔开的正整数(行尾没有多余的空格)，
为从小到大排好序的不相同的随机数。

输入样例

10

20 40 32 67 40 20 89 300 400 15

输出样例

8

15 20 32 40 67 89 300 400

SET应用——SUMX

考虑一组 n 个不同的正整数 $a_1, a_2, \dots, a_m$ ，它们的值在1到1000000之间。给定一个整数 x 。写一个程序sumx计算这样的数对个数 (a_i, a_j) ， $1 \leq i < j \leq n$ 并且 $a_i + a_j = x$ 。

输入要求

标准输入的第一行是一个整数 n ($1 \leq n \leq 1000000$)。第二行有 n 个整数表示元素。第三行是一个整数 x ($1 \leq x \leq 2000000$)。

输出要求

输出一行包含一个整数表示这样的数对个数。

知识点及提示

不同的和为13的数对是(12, 1),

(10, 3)和(2, 11)。

输入样例

9

5 12 7 10 9 1 2 3 11 13

输出样例

3

SET中的ERASE操作

- 用 `erase(x)` 删除元素。
- 其中 `x` 可以是具体的数或迭代器。删除 `set` 中不存在的元素 会被忽略。

```
未命名1.cpp  erase.cpp  输出.cpp
1  #include<bits/stdc++.h>
2  using namespace std;
3  set<int> a={10,20,30,40,50};
4  int main()
5  {
6      set<int>::iterator it=a. begin();
7      ++ it;
8      a. erase(it);
9      for(it=a.begin();it!=a.end();it++)
10         cout<<*it<<" ";
11         cout<<endl;
12         a. erase(40);
13         a. erase(60);
14         for(it=a.begin();it!=a.end();it++)
15             cout<<*it<<" ";
16         cout<<endl;
17         return 0;
18     }
```

```
10 30 40 50
10 30 50
```

```
-----
Process exited after 0.9493 seconds
请按任意键继续. . .
```


SET 中的 FIND 操作

□ `find(x)` 返回 `x` 元素的迭代器，如果找不到 `x` 就返回 `end()` 的迭代器。

```
未命名1.cpp  erase.cpp  输出.cpp
1  #include<bits/stdc++.h>
2  using namespace std;
3  set<int> a={10,20,30,40,50};
4  int main()
5  {
6      set<int>::iterator it;
7      cout<<*a.find(20)<<endl;
8      cout<<*a.find(60)<<endl;
9      return 0;
10 }
```

```
F:\c++\备课\STL\set\erase.exe
20
5
-----
Process exited after 0.8559 seconds
请按任意键继续...
```

SET 中的 COUNT 操作

- `count()` 用来查找 `set` 中某个键值出现的次数。
- 这个函数在 `set` 并不是很实用，因为一个键值在 `set` 只可能出现 0 或 1 次，这样就变成了判断某一键值是否在 `set` 出现过了。

LOWER_BOUND和UP_BOUND

- `lower_bound(x)` 返回 `set` 中大于等于 `x` 的最小元素的迭代器。
- `upper_bound(x)` 返回 `set` 中大于 `x` 的最小元素的迭代器。如果找不到也会返回 `end()` 的迭代器。

```
#include<bits/stdc++.h>
using namespace std;
set<int> a={2,7,8,15,17};
int main()
{
    cout<<*(a.lower_bound(6))<<endl;
    cout<<*(a.lower_bound(7))<<endl;
    cout<<*(a.lower_bound(30))<<endl;
    return 0;
}
```

F:\c
7
7
5

Proc

```
#include<bits/stdc++.h>
using namespace std;
set<int> a={2,7,8,15,17};
int main()
{
    cout<<*(a.upper_bound(6))<<endl;
    cout<<*(a.upper_bound(7))<<endl;
    cout<<*(a.upper_bound(30))<<endl;
    return 0;
}
```

F:\c
7
8
5

Proc

MULTISET

- `multiset`和`set`的定义和成员函数都相同。
- `multiset`和`set`的区别是：`set`插入的元素不能相同，但是`multiset`可以相同。
- 如果删除元素`x`，那么在定义的比较关系下和`x`相等的所有元素都会被删除。
- `count(x)`：`set`能返回 0 或者 1 ， `multiset`是有多少个返回多少个。

MULTISET应用——DEC

给定N个数 A_i ，以及一个正整数C，问有多少对 i, j ，满足 $A_i - A_j = C$ 。

输入要求

第一行输入两个空格隔开的整数N和C

第2至N+1行每行包含一个整数 A_i

输出要求

输出一个数表示答案。

输入样例

5 3

2

1

4

2

5

输出样例

3

PAIR的基本用法

pair的定义

- **pair**是一种模版类型，又称二元组。
- 每个**pair**可以存储两个值。这两种值无限制。也可以将自己写的**struct**的对象放进去，或者说**pair**是一个二元组的结构体，定义完成的**pair**类型可以用在多种场合。
- 定义，例如：
 - `pair<int, char> p;`
 - `pair<string,int> p;`
 - `pair<int ,int > p;`
 - `pair<double,int> p;`
 - `pair<char ,pair<int ,int > > p;`//也可以这样定义一个三元组

pair的使用

□ pair的赋值

- ▣ `pair<int,char>a;`

- ▣ 方式一： `make_pair(233, 'a');`

- ▣ 方式二： `a.first = 233; a.second = 'a';`

- ▣ 这两种方式都可以实现对a的赋值，建议用方式一

□ pair的使用

- ▣ 赋值完成后使用第一关键字要用`a.first`表示,第二关键字用`a.second`表示

- 支持以`first`为第一关键字，`second`为第二关键字的比较函数(`<`, `>`, `<=`, `>=`, `==`, `!=`)

pair应用——买蛋糕2

今天是路路的生日，生日蛋糕自然是少不了。路路的朋友们一起去蛋糕店来买蛋糕，可是等一行人到了蛋糕店之后，发现那里是人山人海啊-_-。这下可把店家给急坏了，因为人数过多，需求过大，所以人们要等好长时间才能拿到自己的蛋糕。由于每位客人订的蛋糕都是不同风格的，所以制作时间也都不同。老板为了最大限度的使每位客人尽快拿到蛋糕，因此他需要安排一个制作顺序，使每位客人的平均等待时间最少。这使他发愁了，于是他请你来帮忙安排一个制作顺序，使得每位客人的平均等待时间最少。

输入要求

输入有两行。第一行是一个整数 n ，表示有 n 种蛋糕等待制作（ $1 \leq n \leq 100$ ）。

第二行有 n 个数，第 i 个数表示第 i 种蛋糕的制作时间。

输出要求

输出包括一行，有 n 个整数，整数间用空格隔开，行末没有空格，是蛋糕的制作顺序，每个数即是蛋糕的编号。

输入样例

8

4 5 3 3 1 4 6 7

输出样例

5 3 4 1 6 2 7 8

MAP&&MULTIMAP基本用法

什么是Map

- Map是STL的一个关联容器，它提供一对一的数据处理能力，map不允许重复元素。
- 其中第一个称为关键字key，每个关键字只能在map中出现一次，第二个称为该关键字的值value，元素的次序由它们的key决定，和value无关。
- map内部自建一颗红黑树，这颗树具有对数据自动排序的功能，所以map根据元素的key自动对元素进行排序。这么一来，根据已知的key搜寻某个元素时，时间复杂度为 $O(1)$ ，而根据已知value搜寻元素时，性能就很糟糕。
- 可起到hash表的作用。

Map的功能

- 自动建立Key - value的对应。key 和 value可以是任意你需要的类型。
- 根据key值快速查找记录。
- 快速插入Key -Value 记录。
- 快速删除记录。
- 根据Key修改value记录。
- 遍历所有记录。

Map的定义

- 需要头文件 `<map>` 或 `<bits/stdc++.h>`
- 构造函数：
`map<Type1,Type2> a;`
- `map`对象是模板类，需要关键字和存储对象两个模板参数。
- 例如 `map<int,string> a;` 这样就定义了一个用`int`作为索引,并拥有相关联的指向`string`的指针。右图所有的定义都是可行的。

```
1  #include<bits/stdc++.h>
2  using namespace std;
3  map<int,int> a;
4  map<string,int>b;
5  map<string,string>c;
6  map<char,string>d;
7  map<char,double>e;
8  map<double,string>f;
9  int main()
10 {
11     return 0;
12 }
```

Map的赋值

- Map赋值的两种方法：
 - ▣ 用insert函数插入pair数据
 - ▣ 用数组方式插入数据。
- 以上两种用法的区别：
 - ▣ 用insert函数插入数据，在数据的插入上涉及到map关键字的唯一性这个概念。即当map中有这个关键字时，insert操作无效的。
 - ▣ 用数组方式它可以覆盖以前该关键字对应的值。
- 时间复杂度皆为 $O(\log n)$ 。

未命名3.cpp

```
1 #include<bits/stdc++.h>
2 using namespace std;
3 map<int,string> a;
4 int main()
5 {
6     a.insert(make_pair(1,"a"));
7     a[2]="b";
8     cout<<a[1]<<endl;
9     cout<<a[2]<<endl;
10    return 0;
11 }
```

未命名3.cpp

```
1 #include<bits/stdc++.h>
2 using namespace std;
3 map<int,string> a;
4 int main()
5 {
6     a.insert(make_pair(1,"a"));
7     a.insert(make_pair(2,"b"));
8     a[2]="c";
9     cout<<a[1]<<endl;
10    cout<<a[2]<<endl;
11    return 0;
12 }
```

Map的Begin、End与iterator

- map的迭代器(iterator)是一种检查容器内元素并遍历元素的数据类型。
- Begin(), End()是map的成员函数，返回值分别是map中**首个元素**的迭代器和**末尾元素向下一位**的迭代器。
- map的迭代器仅支持++和--。
- 时间复杂度 $O(1)$

未命名1.cpp

```
1 #include<bits/stdc++.h>
2 using namespace std;
3 map<int,string> a;
4 int main()
5 {
6     a.insert(pair<int,string>(1,"a"));
7     a.insert(make_pair(2,"b"));
8     a[3]="c";
9     map<int,string>::iterator it;
10    for(it=a.begin();it!=a.end();it++)
11        cout<<it->first<<" "<<it->second<<" ";
12    cout<<endl;
13    return 0;
14 }
15
```

G:\C++\STL\map\未命名1.exe

1 a 2 b 3 c

Process exited after 2.66 seconds with return value 0
请按任意键继续. . .

Map的输出

- **map**的单个**value**可以像数组那样使用下标输出。
- 要输出所有元素需要使用迭代器依次遍历。
- 输出 **it->first** 对应的关键字 **key**
- 输出 **it->second** 对应的为其 **value**。

未命名1.cpp

```
1 #include<bits/stdc++.h>
2 using namespace std;
3 map<int,string> a;
4 int main()
5 {
6     a.insert(pair<int,string>(1,"a"));
7     a.insert(make_pair(2,"b"));
8     a[3]="c";
9     cout<<a[1]<<endl;
10    cout<<a[2]<<endl;
11    cout<<a[3]<<endl;
12    return 0;
13 }
```



C:\Documents and Settings\...
a
b
c

Process exited after 0.2
请按任意键继续. . .

```
#include<bits/stdc++.h>
using namespace std;
map<int,string> a;
int main()
{
    a.insert(pair<int,string>(1,"a"));
    a.insert(make_pair(3,"b"));
    a[5]="c";
    map<int,string>::iterator it;
    for(it=a.begin();it!=a.end();it++)
        cout<<it->first<<" "<<it->second<<" ";
    cout<<endl;
    return 0;
}
```



G:\C++\STL\map\未命名1.exe

1 a 3 b 5 c

Map的rbegin() 和rend()

- Map 中的元素总是保持单调递增。
- begin() 返回的迭代器指向map 中的最小key值；
- rbegin() 返回的迭代器指向 map 中的最大key值。
- Map 中的元素总是保持单调递增。
- end() 返回的迭代器指向 Map 中的最后元素的后一个位置；
- rend() 返回指向Map中第一个元素的反向迭代器

Map的反序遍历

```
#include<bits/stdc++.h>
using namespace std;
map<int,string> a;
int main()
{
    a.insert(pair<int,string>(1,"a"));
    a.insert(make_pair(3,"b"));
    a[5]="c";
    map<int,string>::reverse_iterator rit;
    for(rit=a.rbegin();rit!=a.rend();rit++)
        cout<<rit->first<<" "<<rit->second<<" ";
    cout<<endl;
    return 0;
}
```

H:\C++\STL\map\未命名2.exe

5 c 3 b 1 a

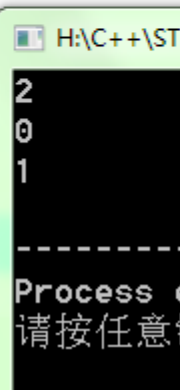
Process exited after 0.04522 seconds with return value 0
请按任意键继续. . .

map 中 size、empty 和 clear

- **Size()** 是 **map** 的成员函数，其返回值一个无符号整数，表示 **map** 中元素的个数。时间复杂度 $O(1)$ 。
- **empty()** 返回一个 **bool** 类型，表示 **map** 是否为空。时间复杂度 $O(1)$ 。
- **clear()** 清除 **Map** 中的所有元素。时间复杂度 $O(\text{size})$ 。

未命名3.cpp

```
3 map<int,string> a;  
4 int main()  
5 {  
6     a.insert(make_pair(1,"a"));  
7     a.insert(make_pair(2,"b"));  
8     a[2]="c";  
9     cout<<a.size()<<endl;  
10    cout<<a.empty()<<endl;  
11    a.clear();  
12    cout<<a.empty()<<endl;  
13    return 0;  
14 }
```



Map应用——count

有 n 个自然数，每个数均不超过1500000000
(1.5×10^9)。已知不相同的数不超过10000个，
现在需要统计这些自然数各自出现的次数，并按
照自然数从小到大的顺序输出统计结果。

【输入要求】

输入包含 $n+1$ 行：

第1行是整数 n ，表示自然数的个数。

第2到 $n+1$ 行每行一个自然数。

【输出要求】

从小到大输出若干行，每行为一个数字和它出现的
次数。

【解题提示】 $n \leq 300000$

【输入样例】

```
8
2
4
2
4
5
100
2
100
```

【输出样例】

```
2 3
4 2
5 1
100 2
```

Map应用——drawer

期末考试即将来临，小T由于同时肩负了学习、竞赛、班团活动等多方面的任务，一直没有时间好好整理他的课桌抽屉，为了更好地复习，小T首先要把课桌抽屉里的书分类整理好。小T的抽屉里堆着N本书，每本书的封面上都印有学科名称，学科名称用一个字符串表示，如语文学科的书封面上都印有“chinese”。现在，你的任务是帮助小T找出哪个学科的书最多？

输入数据

入文件第一行包含一个自然数N ($0 < N \leq 1000$) 表示抽屉中书的总数。接下来N行每行包含一本书的学科名称，学科名称是一个长度不超过15的由小写英文字母组成的字符串。

输出数据

出文件仅有一行包含一个字符串，表示最多的那种书的学科名称。数据保证答案一定是唯一的。

【输入样例】

5

english

chinese

physics

chinese

chinese

【输出样例】

chinese

Map应用——exclusive

有这样的一个集合，集合中的元素个数由给定的 N 决定，集合的元素为 N 个不同的正整数，一旦集合中的两个数 x, y 满足 $y = P * x$ ，那么就认为 x, y 这两个数是互斥的，现在想知道给定的一个集合的最大子集满足两两之间不互斥。

【输入要求】

第一行给定两个数 N 和 P 。

接下来一行包含 N 个不同正整数 a_i 。

【输出要求】

输出一行表示最大的满足要求的子集的元素个数。

【解题提示】

$1 \leq N \leq 10^5, 1 \leq P \leq 10^9, 1 \leq a_i \leq 10^9$

【输入样例】

4 2

1 2 3 4

【输出样例】

3

Map 中的 count

- `count()` 用来查找Map中某个某个键值出现的次数，这个函数在Map只可能出现0或1次。

```
#include<bits/stdc++.h>
using namespace std;
map<int,string> a;
int main()
{
    a.insert(make_pair(1,"a"));
    a.insert(make_pair(2,"b"));
    a[5]="c";
    cout<<a.count(1)<<endl;
    cout<<a.count(3)<<endl;
    return 0;
}
```



Map 中的 find()

- `find(x)` 返回 `x` 元素的迭代器，如果找不到 `x` 就返回 `end()` 的迭代器。时间复杂度皆为 $O(\log n)$

未命名3.cpp

```
1  #include<bits/stdc++.h>
2  using namespace std;
3  map<int,string> a;
4  int main()
5  {
6      a.insert(make_pair(1,"a"));
7      a.insert(make_pair(2,"b"));
8      a[5]="c";
9      map<int,string>::iterator it,it1;
10     it=a.find(1);it1=a.find(3);
11     cout<<it->first<<it->second<<endl;
12     if(it1==a.end())cout<<"no"<<endl;
13     return 0;
14 }
```

H:\C++\STL\map\未命名3.exe

1a
no

Process exited after 0.06306 sec
请按任意键继续. . .

lower_bound和upper_bound

- `lower_bound(x)` 返回 Map 中key大于等于 `x` 的最小元素的迭代器，时间复杂度皆为 $O(\log n)$ 。
- `upper_bound(x)` 返回 Map 中key大于 `x` 的最小元素的迭代器。如果找不到也会返回 `end()` 的迭代器。

未命名4.cpp

```
3 map<int,string> a;  
4 int main()  
5 {  
6     a.insert(make_pair(2,"a"));  
7     a.insert(make_pair(3,"b"));  
8     a[5]="c";  
9     map<int,string>::iterator it,it1;  
10    it=a.lower_bound(2);  
11    it1=a.upper_bound(3);  
12    cout<<it->first<<it->second<<endl;  
13    cout<<it1->first<<it1->second<<endl;  
14    return 0;  
15 }
```

H:\C++\STL\map\未命名4.exe

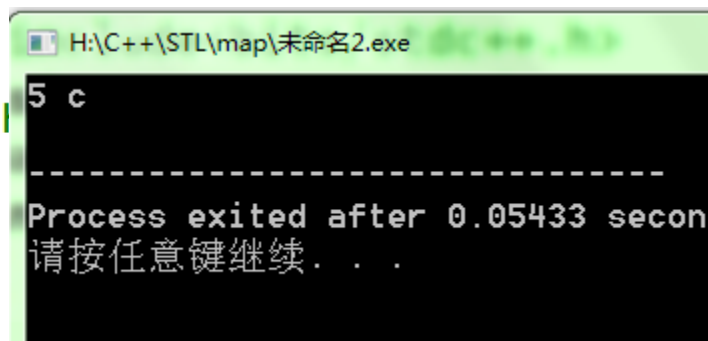
2a
5c

Process exited after 0
请按任意键继续. . .

Map 中的 erase 操作

- 用 `erase(x)` 删除一个元素。
- 其中 `x` 可以是具体的数或迭代器。删除 Map 中不存在的元素会被忽略。
- `a.erase(begin(),end())` 效果和 `a.clear()` 相同
- 时间复杂度 $O(n)$

```
#include<bits/stdc++.h>
using namespace std;
map<int,string> a;
int main()
{
    map<int,string>::iterator it;
    a.insert(pair<int,string>(1,"a"));
    a.insert(make_pair(3,"b"));
    a[5]="c";
    a.erase(1);
    a.erase(2);
    a.erase(a.begin());
    for(it=a.begin();it!=a.end();it++)
        cout<<it->first<<" "<<it->second<<endl;
    return 0;
}
```



Map应用——match

n 个不同的字符串，每个字符串对应一个数字。 q 次询问一个字符串对应什么数字。

【输入要求】

第1行 n, q 。

第2到 $n+1$ 行，每行一个字符串和一个数字，中间用一个空格隔开。

第 $n+2$ 到 $n+q+1$ 行，每行一个询问的字符串。

【输出要求】

q 行，每行一个数字

【解题提示】

$n, q \leq 20000$ 每个字符串的长度 ≤ 30

【输入样例】

5 3

fs3fwe 3

4838fdeewerwer 54

irjfhid 888

847hhhh 1

0000 0

0000

847hhhh

fs3fwe

【输出样例】

0

1

3

Map应用——game

给出了N个单词，已经按长度排好了序。如果某单词i是某单词j的前缀，i->j算一次接龙(两个相同的单词不能算接龙)。

你的任务是：对于输入的单词，找出最长的龙。

【输入要求】

第一行为N。以下N行每行一个单词（由小写组成），已经按长度排序。

【输出要求】

仅一个数，为最长的龙的长度。

【解题提示】

N≤50000 每个单词长度≤50

【输入样例】

```
5
i
a
int
able
inter
```

【输出样例】

```
3
```

MULTIMAP基本用法

multimap 简介 & 功能

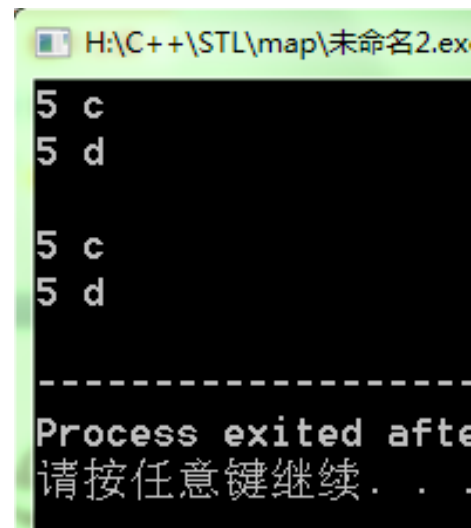
- **multimap** 与 **map** 类似，操作大部分也与 **map** 一样。
- 所不同的是它允许重复键。即一个关键词可以有很多不同的值。这些值按插入的时间顺序排列。

multimap和map的区别

- 在multimap中没有定义[]运算符，因此，multimap进行插入的时候，只能利用insert()函数进行插入。
- 使用find(x)查找返回每种关键词的所有元素的第一个元素的迭代器。
- 指定关键词的遍历：
- equal_range(x)返回一对iterator的pair，表示关键词x的位置的区间（左闭右开）。这个函数其实map也有。
- 也可用lower_bound和upper_bound实现。

equal_range(x)的使用

```
#include<bits/stdc++.h>
#define it map<int,string>::iterator
using namespace std;
multimap<int,string> a;
int main()
{
    a.insert(pair<int,string>(1,"a"));
    a.insert(make_pair(3,"b"));
    a.insert(make_pair(5,"c"));
    a.insert(make_pair(5,"d"));
    a.insert(make_pair(7,"x"));
    pair <it,it> r=a.equal_range(5);
    for(it i=r.first;i!=r.second;i++)
        cout<<i->first<<" "<<i->second<<endl;
    cout<<endl;
    for(it i=a.lower_bound(5);i!=a.upper_bound(5);i++)
        cout<<i->first<<" "<<i->second<<endl;
    return 0;
}
```



The screenshot shows a Windows command prompt window titled "H:\C++\STL\map\未命名2.exe". The output of the program is displayed as follows:

```
5 c
5 d
5 c
5 d
-----
Process exited after
请按任意键继续. . .
```

The output demonstrates that the `equal_range(5)` function correctly identifies the range of elements with the key 5, which are the pairs (5, "c") and (5, "d").

Multimap应用——double

n条信息，一条信息包括一个字符串**s**和一个整数**x**，表示名字为**s**的奶牛获得了一棵质量为**x**的草。一头奶牛可能获得不止一棵草，但不超过十棵草。

q次询问一头奶牛获得了哪些质量的草，按输入顺序先后输出。

【输入要求】

一行一个整数**n**。

n行每行一个字符串**s**和一个整数**x**，以空格隔开。

一行一个整数**q**。

q行每行一个字符串**s**，表示询问的奶牛的名字。

【输出要求】

对于每个询问输出若干行，每一行一个整数，表示此奶牛**s**获得的草的质量，按输入顺序输出。若**s**一棵草都没有，则不输出。

【输入样例】

```
3
a 2
a 1
b 29
3
b
a
c
```

【输出样例】

```
29
2
1
```

DEQUE双端队列基本用法

什么是deque?

- deque (double-ended queue), 顾名思义, 是一种双端队列。也是动态数组的一种形式。
- 需要包含头文件 `<bits/stdc++.h>` 或头文件 `<queue>`
- deque 的大小是动态的。支持在头部或尾部快速插入/删除、快速随机访问、在任意位置的插入/删除等操作。

queue 与 vector 的对比

- 都支持 $O(1)$ 复杂度的尾部操作和随机访问[],但 **deque** 的随机访问常数更大。
- **deque** 额外支持 $O(1)$ 复杂度的头部操作 **push**、**pop**。
- 在中间位置的操作复杂度都为 $O(\text{size})$ 。
- 所以如果只是当普通数组使用, **vector** 速度更快;但如果要频繁在头、尾部操作, **deque** 拥有更好的效率。

构造函数

- **deque** 的构造与 **vector** 非常类似，其类型也不限。完全可以当做 **vector** 定义、使用。

```
deque <int> a;  
// 定义一个类型为 int、初始为空的 deque  
deque <int> b(5);  
// 定义一个类型为 int、初始大小为 5 的 deque  
// 每个元素都是 0  
swap(a, b);  
// 交换了 a 和 b 的值  
deque < vector <int> > c;  
// 其类型不限，甚至可以是 vector
```

= 运算符

- 普通的赋值运算。时间复杂度 $O(\text{size})$

```
deque <int> a(3); // 初始时 a 包含 3  
个 0 deque <int> b(5); // 初始时 b 包  
含 5 个 0  
b = a; // 此时 b 包含 3 个 0  
a = deque <int>(); // 此时 a 为空
```

[] 运算符

- 如普通数组或 **vector** 一样，直接访问该下标的元素。时间复杂度 $O(1)$ ，但常数比 **vector** 大。注意其下标是从 **0** 开始的。

```
deque <int> q(5);  
for (int i = 0; i < 5; ++i)  
    cout << q[i] << endl; // 0 0 0 0  
0  
  
for (int i = 0; i < 5; ++i)  
    q[i] = i; // 0 1 2 3 4
```

front 函数和 back 函数

- 分别返回队头元素或队尾元素的值。时间复杂度 $O(1)$

```
deque < long long > q(10);  
for ( int i = 0; i < 10; ++ i)  
    q[i] = i;  
cout<<q. front ()<<' '<<q. back ()<< endl;  
// 0 9
```


size() & empty() 操作

□ empty()

判断是否为空，1为空，0为不空，时间复杂度 $O(1)$

□ size()

返回队列**deque**中元素个数，时间复杂度 $O(1)$

```
deque <int> q(10);  
cout << q.empty() << endl; // 0  
q = deque <int>(); //q.resize(0)  
cout << q.empty() << endl; // 1
```

deque的Begin、End与iterator

- deque可用[]遍历，只有在进行 insert 或erase 等特定操作时才必须使用迭代器(iterator)。
- deque的迭代器支持运算 ++, --, +=, -=。
- Begin(), End()是deque的成员函数，返回值分别是deque中首个元素的迭代器和deque中末尾元素向后的迭代器。
- 以上操作时间复杂度均为 $O(1)$ 。

deque的Clear操作

- `Clear()`是deque的成员函数，使用后将deque整个清空。
- 无参数。清除 deque 中所有元素，释放所有空间。时间复杂度 $O(n)$ 。

```
#include<bits/stdc++.h>
using namespace std;
deque<int> q;
int main()
{
    int i;
    for(i=1;i<=5;i++)
        q.push_back(i);
    for(i=0;i<q.size();i++)
        cout<<q[i]<<" ";
    cout<<endl;
    cout<<q.size()<<endl;
    q.clear();
    cout<<q.size()<<endl;
    for(i=0;i<q.size();i++)
        cout<<q[i]<<" ";
    cout<<endl;
    return 0;
}
```

F:\c++\课\STL

1 2 3 4 5
5
0

Process exited
请按任意键继续

队头和队尾的操作

- `push_front(val)`, 在队头插入 `val`。 `push_back(val)`, 在队尾插入 `val`。 `pop_front()`, 删除队头元素。
- `pop_back()`, 删除队尾元素。
- 时间复杂度均为 $O(1)$ 。

```
deque <int> q(2); // 0 0
q.push_back(200); // 0 0 200
q.push_front(0); // 0 0 0 200
q.push_front(300); // 300 0 0 0 200
q.pop_back(); // 300 0 0 0
q.pop_front(); // 0 0 0
```

deque的应用

- 由于 **deque** 的空间是动态的，可以代替循环队列，实现 **SPFA** 算法(**Shortest Path Faster Algorithm** 最短路快速算法)队列实现。
- 在开启 **O2** 优化后速度与手写循环队列相差无几。
- 接下来着重介绍一下单调队列。

单调队列引入——老徐真人秀

- 最美杭城人老徐参加一个真人秀，节目共录制 N 天，每一天节目组都提供同一种美味的苹果若干，老徐有个怪癖，每一天只会吃天数不超过 M ($1 \leq M \leq N \leq 10^6$) 天的最美味苹果。
- 老徐是编程达人，决定通过程序来快速选择。请老徐回答当时是怎么做到的？
- 老徐是大师，当然不会直接告诉你答案，当时随手扔过来下面这个：
- 比如， $N = 5$ ， $M = 4$ ，5天提供的苹果美味度分别为(80,75,78,73,79)，老徐的做法如下：



单调队列应用——滑动窗口(最大值)

给出一个长度为 n 的数组($n \leq 1000$)，有一个可移动的长度为 k 的窗口，从最左端开始每个时刻向右移动一格，你的任务是输出每个时刻窗口内最大及最小的数字。

Window position	Maxvalue
[1 3 -1] -3 5 3 6 7	3
1 [3 -1 -3] 5 3 6 7	3
1 3 [-1 -3 5] 3 6 7	5
1 3 -1 [-3 5 3] 6 7	5
1 3 -1 -3 [5 3 6] 7	6
1 3 -1 -3 5 [3 6 7]	7

输入样例

8 3

1 3 -1 -3 5 3 6 7

输出样例

3 3 5 5 6 7

输入要求

一行包含四个整数 n, k, p, q ，满足 $a[0]=1, a[i]=(a[i-1]*p+i) \% q$

输出要求

一个数值，表示每个时刻窗口内最大值的和。

滑动窗口——单调队列解法

- 先只考虑如何求最大值。
- **1 3 -1 -3 5 3 6 7**
- 观察一下这组数据。当 **5** 被加入窗口后，如果之后的窗口能覆盖到 **-1** 或 **-3**，也一定可以覆盖到 **5**。因此之后的最大值不可能是 **-1** 或 **-3**。
- 从而可以得出一个结论：若窗口中存在 a ，当前加入一个数 b 满足 $b > a$ ，那么 a 已经没有保留的必要。
- 如果后面的数比前面的大，那么前面的数不可能成为最大值。如果后面的数比前面的小，那么两个数都可能成为最大值。
- 进而产生一种想法：维护一个队列存储可能成为最大值的数，同时保证任何时刻队列元素单调减。

滑动窗口——单调队列解法

- 每次窗口向后移动会加入一个新的数，从队尾不断删去比其小的数，然后把这个数插入队尾。
- 同时判断如果队首元素不在当前窗口内则删去队首。队首元素就是当前窗口的最大值。

窗口位置	单调队列中的元素	
<u>1 3</u> -1 -3 5 3 6 7	3 -1	1先入队，3比1年轻且大，故1被挤出，-1，-3都比3年轻故入队
1 <u>3 -1 -3</u> 5 3 6 7	3 -1 -3	
1 3 <u>-1 -3</u> 5 3 6 7	5	5比所有人都年轻，故挤掉所有人入队，3比5年轻故入队
1 3 -1 <u>-3 5</u> 3 6 7	5 3	
1 3 -1 -3 <u>5 3</u> 6 7	6	6、7都比所有人都年轻，故挤掉所有人入队
1 3 -1 -3 5 <u>3 6</u> 7	7	

任何时刻，队列中的数都是从大到小单调排列的，最大值即堆头元素

滑动窗口复杂度分析与实现

- 由于每个元素各入队、出队一次，时间复杂度为 $O(n)$ 。用 **deque** 简洁地实现，代码量非常小，队列里存放的是元素的位置。

```
for ( int i = 1; i <= n; ++ i)
{
    while (! q.empty() && a[i] > a[q.back()])
        q.pop_back(); //删去队尾比其小的数
    q.push_back(i); //把当前位置插入队尾
    while (q.front() <= i-k)
        q.pop_front(); //删去窗口外的数
    ans[i] = a[q.front()]; //队首即为答案
}
```

单调队列应用——滑动窗口(最小值)

给出一个长度为 n 的数组($n \leq 1000$), 有一个可移动的长度为 k 的窗口, 从最左端开始每个时刻向右移动一格, 你的任务是输出每个时刻窗口内最大及最小的数字。

Window position	Minvalue
[1 3 -1] -3 5 3 6 7	-1
1 [3 -1 -3] 5 3 6 7	-3
1 3 [-1 -3 5] 3 6 7	-3
1 3 -1 [-3 5 3] 6 7	-3
1 3 -1 -3 [5 3 6] 7	3
1 3 -1 -3 5 [3 6 7]	3

输入要求

一行包含四个整数 n, k, p, q , 满足 $a[0]=1, a[i]=(a[i-1]*p+i) \% q$

输出要求

一个数值, 表示每个时刻窗口内最小值的和。

输入样例

8 3

1 3 -1 -3 5 3 6 7

输出样例

-1 -3 -3 -3 3 3

deque应用——advertisement

最近，**afy**决定给**TOJ**印刷广告，广告牌是刷在城市的建筑物上的，城市里有紧靠着的**N**个建筑。**afy**决定在上面找一块尽可能大的矩形放置广告牌。我们假设每个建筑物都有一个高度，从左到右给出每个建筑物的高度**H1,H2...HN**，且 **$0 < H_i \leq 10^8$** ，并且我们假设每个建筑物的宽度均为**1**。要求输出广告牌的最大面积。

输入要求

第一行：一个数**n** ($n \leq 1000$)。

第二行：**n**个数,给出这**n**个建筑物。

输出要求

第一行：一个数**ans**，表示最大面积。

知识点及提示:一个高为**4**，长度为**6**的广告牌。

输入样例

6

5 8 4 4 8 4

输出样例

24

advertisement 问题分析

- 枚举每一座建筑，假设当前枚举的建筑是作为广告牌的最高高度。
- 问题转化为求出每座建筑左边第一个比其矮的建筑和右边第一个比其矮的建筑。
- 正反两次扫描，维护一个单调队列。如果新加入的建筑比队尾的建筑矮，那么队尾建筑就只能拓展到新加入建筑的位置。
- 对于边界情况，可以强制在最左边与最右边加入一个非常矮的建筑。
- 时间复杂度 $O(n)$ 。

deque应用——最大全1子矩阵

输入一个 $n*m$ 的01矩阵，求最大全1子矩阵的面积。

输入要求

输入第一行有两个用一个空格隔开的正整数 n, m 表示矩阵的大小。

接下来 n 行，每行 m 个01字符，每两个字符间用一个空格隔开。

输出要求

输出一个整数，表示最大面积。

知识点及提示： $n, m \leq 1000$

可以借鉴advertisement的思路

输入样例

4 4

1 1 1 1

1 0 0 1

1 0 0 1

1 1 1 1

输出样例

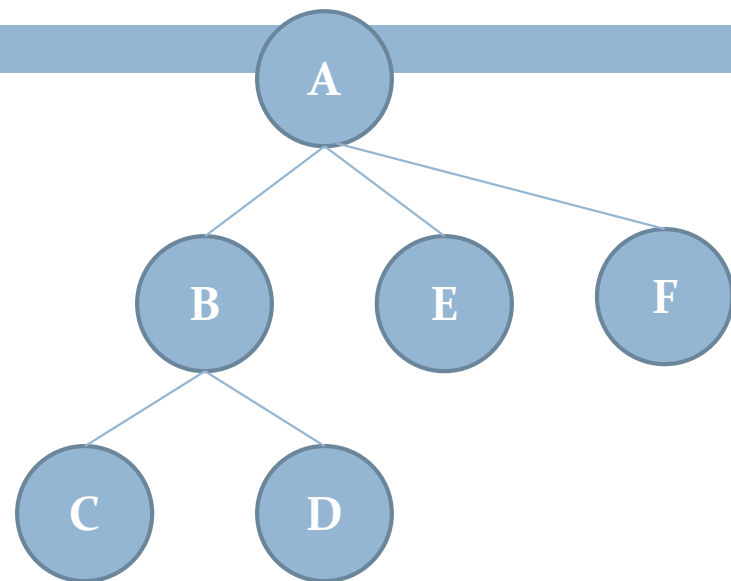
4

PRIORITY_QUEUE基本用法

树

树形结构是一类非常重要的**非线性结构**，在我们的程序设计中，也有这样一种数据结构，它的长相和

右图非常相似。因此，我们就借用生活中的观念，把这样的结构称为“树”，它可以很好地描述客观世界中广泛存在的具有**分支关系或层次特性**的对象。



二叉树

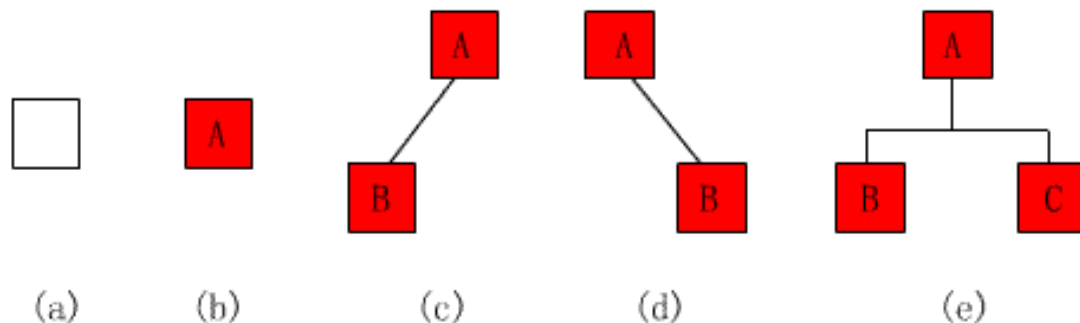
什么是二叉树呢？用上面的定义来说的话，就是树的度不超过2的树结构，即任何一个结点的儿子数量不超过2。

在二叉树中，每个结点都有明确的左儿子和右儿子。因此，每棵二叉树都是有序树。

二叉树

二叉树的基本形态：

二叉树也是递归定义的，其结点有左右子树之分，逻辑上二叉树有五种基本形态

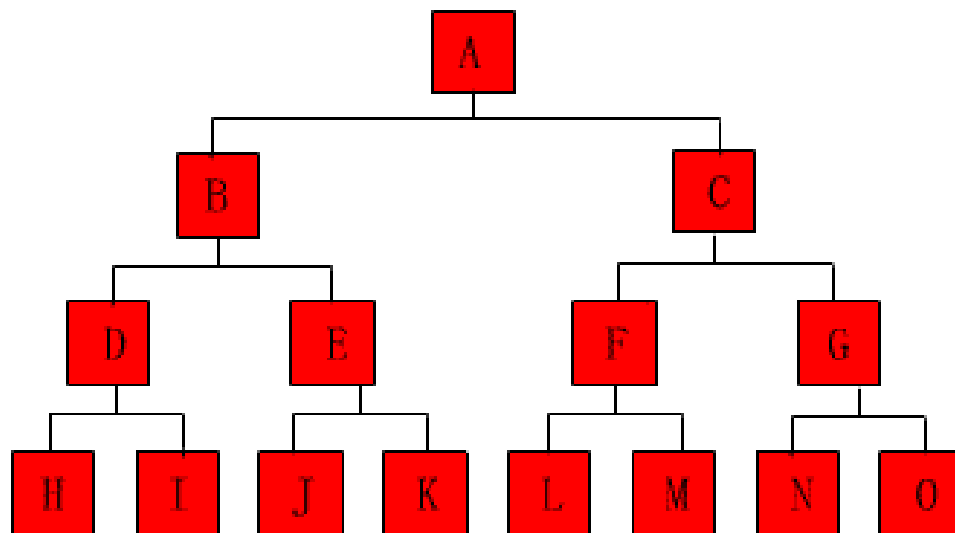


(a)空二叉树；(b)只有一个根结点的二叉树；(c)右子树为空的二叉树
(d)左子树为空的二叉树；(e)完整二叉树

满二叉树

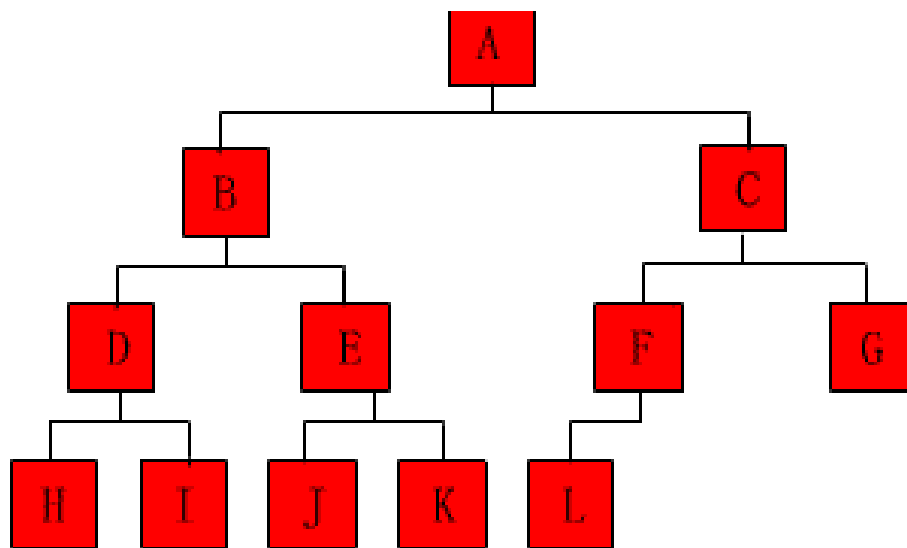
在二叉树大家族中，还有几种特殊的二叉树值得我们注意。

一种是除了叶结点外每一个结点都有左右子女且叶结点都处在最底层的二叉树。我们称之为**满二叉树**。



完全二叉树

另一种是只有最下面的两层结点的度小于2，并且最下面一层的结点都集中在该层最左边的若干位置的二叉树。我们称之为**完全二叉树**。

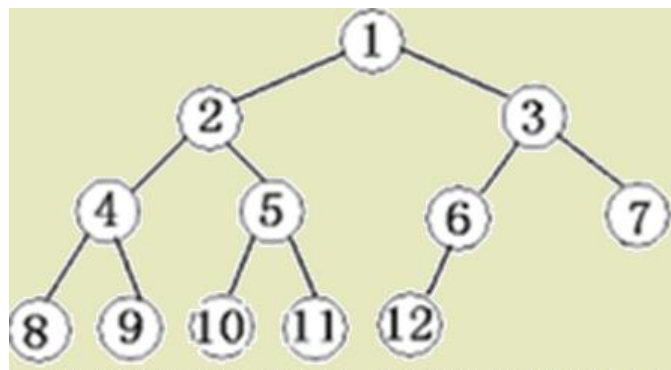


完全二叉树的性质

一棵 n 个结点的完全二叉树的深度为： $\text{floor}(\log_2(n))+1$ 。

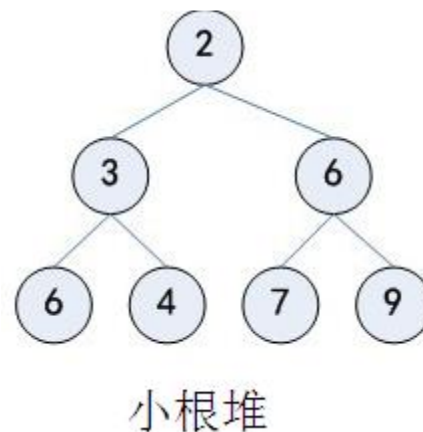
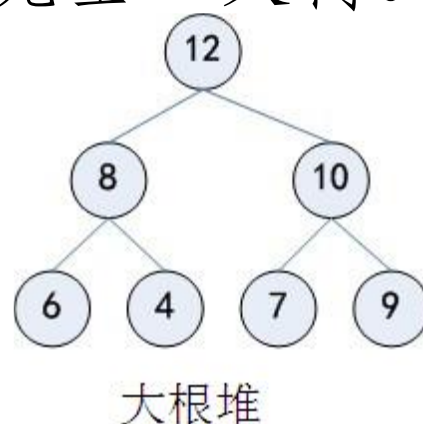
一棵 n 个结点的完全二叉树，对于编号为 i 的结点一定满足：

1. 如果 $i=1$ ，则结点 i 为根，无父结点；如果 $i>1$ ，则其父结点编号为 $i/2$ ；
2. 如果 $2*i>n$ ，则结点 i 为叶结点；否则左孩子编号为 $2*i$ ；
3. 如果 $2*i+1>n$ ，则结点 i 无右孩子；否则右孩子编号为 $2*i+1$ 。



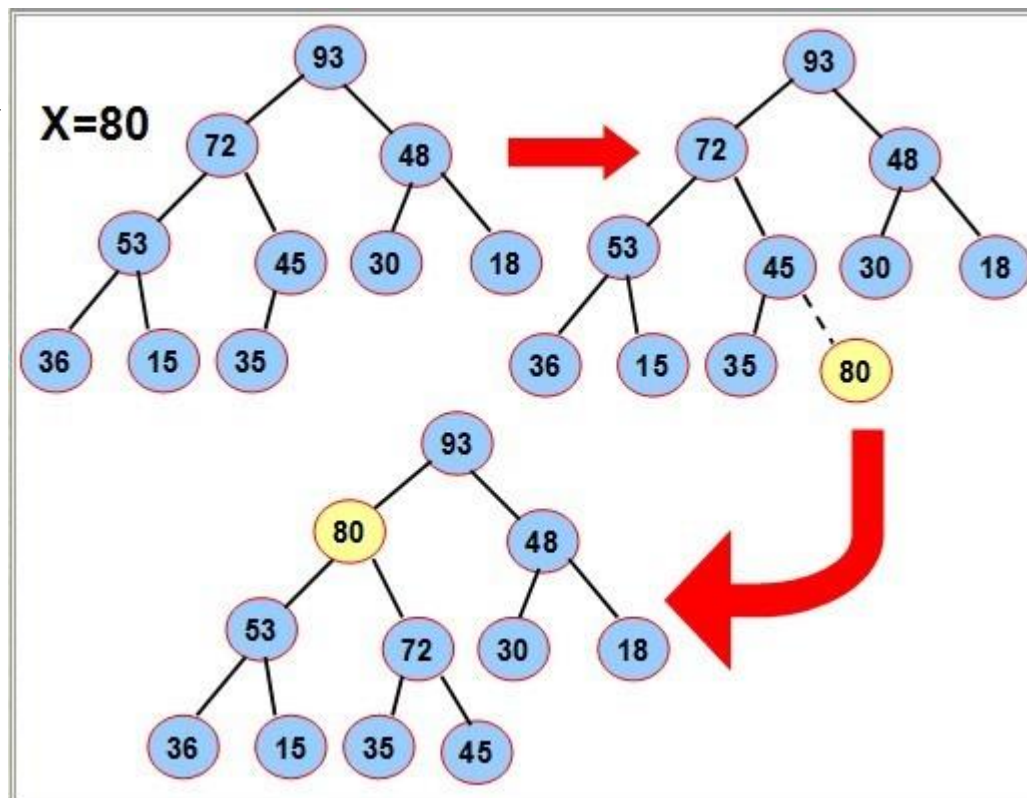
堆的基本概念

- 堆是一类特殊的数据结构，常用的堆为二叉堆，形式上是一个数组，本质上是一棵完全二叉树。
- 二叉堆有大根堆和小根堆之分。
 - **大根堆**是一棵除根结点以外的每个结点 i ，键值都大于等于其孩子结点键值的完全二叉树 ($A[\text{parent}(i)] \geq A[i]$)。
 - **小根堆**是一棵除根结点以外的每个结点 i ，键值都小于等于其孩子结点键值的完全二叉树 ($A[\text{parent}(i)] \leq A[i]$)。
 - 堆中的任一子树也还是堆。



堆的基本操作

- **push**操作(插入)：往堆尾加入一个元素，并通过从下往上调整法，使其继续保持堆的性质。
- **get**操作（删除）：从堆中取出堆头元素，并删除该结点（堆尾覆盖），再通过从上往下的调整法，使其继续保持堆的性质



priority_queue简介

- priority_queue（优先队列） 是一个拥有权值概念的单向队列（queue），在这个队列中，所有元素是按优先级排列的。
- 需要头文件<bits/stdc++.h>或头文件<queue>
- 在STL的具体实现中，priority_queue也是以别的容器作为底部结构，再根据堆的处理规则来调整元素之间的位置。
- 可当作堆用。

priority_queue定义&赋值

- 定义：

- `priority_queue<Type, Container, Functional>`

- **Type**为数据类型， **Container**为保存数据的容器，**Functional**为元素比较方式。

- 第一个参数必须写，后两个可不写。

- 如果不写后两个参数，那么容器默认用的是**vector**，比较方式默认用**operator<**，也就是大根堆（维护最大值）。

- 即与**`priority_queue<int,vector<int>,less<int>>`**效果相同。

priority_queue定义&赋值

- 若要使用一般的小根堆（维护最小值），可以写
`priority_queue<int,vector<int>,greater<int>>`
- 赋值与**vector**不同，仅可赋为同类型的变量。

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main(){
5     priority_queue<int> q; //大根堆
6     priority_queue<int,vector<int>,greater<int>> p; //小根堆
7     priority_queue<int> a={1,2}; // Wrong!
8     priority_queue<int> b=q; // Accepted
9     return 0;
10 }
```

push() & top() & pop() 操作

□ push(Elem e)

- ▣ 向优先队列里增加一元素
- ▣ 时间复杂度为 $O(\log n)$

□ pop()

- ▣ 队列头部数据（最值）出队
- ▣ $O(\log n)$

□ top()

- ▣ 返回头部数据（最值）
- ▣ $O(1)$

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main(){
5     priority_queue<int> q;
6     priority_queue<int> p=q;
7     q.push(1);
8     q.push(3);
9     q.push(2);
10    printf("%d\n",q.top());
11    q.pop();
12    printf("%d\n",q.top());
13    return 0;
14 }
```

- 输出:
- 3
- 2

size() & empty() 操作

□ empty()

- ▣ 判断是否为空，1为空，0为不空，时间复杂度 $O(1)$

□ size()

1. 返回优先队列中元素个数，时间复杂度 $O(1)$

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main(){
5      priority_queue<int> q;
6      q.push(1);
7      q.push(3);
8      cout << q.empty() << '\n';
9      cout << q.size() << '\n';
10     q.pop();
11     q.pop();
12     cout << q.empty() << '\n';
13     cout << q.size() << '\n';
14     return 0;
15 }
```

• 输出:

- 0
- 2
- 1
- 0

Hint

- `priority_queue` 不支持对于头部以外元素的访问和操作，所以不能用迭代器。
- 无 `clear()` 操作，只能手写。如右图。
- 访问 `top()` 时，注意队列是否为空。

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  priority_queue<int> q;
4
5  void clear(){
6      while (!q.empty())
7          q.pop();
8  }
9
10 int main(){
11     q.push(1);
12     clear();
13     cout << q.size() << '\n';
14     return 0;
15 }
```

priority_queue应用——懒羊羊吃草

懒羊羊也有存储粮食的习惯。每当他存储一份粮食时，他会专门拿出一个筐来存放。因此，他的仓库里有很多很多筐的青草。而我们的懒羊羊又是一个经常馋嘴的小羊，每当他想吃草时，就会从仓库里找出数量最少的一筐草，把它吃掉。可是懒羊羊因为草吃得太多了导致大脑运转缓慢，所以他不得不向你请求支援，帮他找出他应该吃数量为多少的青草。

输入要求

第一行为一个正整数 n ，表示懒羊羊一共进行了 n 次操作
($2 \leq n \leq 1000000$)

第二行至第 $n+1$ 行每行表示一个懒羊羊的操作，当这行形式为 单独一个字符' q '时，表示懒羊羊肚子饿了，要吃掉仓库里当前数量最少的那份青草；当这行形式为一个字符' i '和一个整数 k 时，表示懒羊羊将一份数量为 k 的青草存入了仓库，' i '和 k 之间用空格隔开。

输入数据保证每次询问时仓库里都有草可吃且所有操作中懒羊羊至少会吃一次草。

输出要求

每当输入为' q '时，输出懒羊羊当前吃掉的那份青草的数量是多少。

输入样例

```
5
i 5
i 2
q
i 9
q
```

输出样例

```
2
5
```

priority_queue应用——合并果子

(NOIP2004高中组第2题)

在一个果园里，多多已经将 n 个果子打了下来，而且按果子的不同种类分成了不同的堆。多多决定把所有的果子合成一堆。

每一次合并，多多可以把两堆果子合并到一起，消耗的体力等于两堆果子的重量之和。可以看出，所有的果子经过 $n-1$ 次合并之后，就只剩下一堆了。多多在合并果子时总共消耗的体力等于每次合并所耗体力之和。

因为还要花大力气把这些果子搬回家，所以多多在合并果子时要尽可能地节省体力。假定每个果子重量都为1，并且已知果子的种类数和每种果子的数目，你的任务是设计出合并的次序方案，使多多耗费的体力最少，并输出这个最小的体力耗费值。

例如有3种果子，数目依次为1，2，9。可以先将1、2堆合并，新堆数目为3，耗费体力为3。接着，将新堆与原先的第三堆合并，又得到新的堆，数目为12，耗费体力为12。所以多多总共耗费体力 $=3+12=15$ 。可以证明15为最小的体力耗费值。

输入要求

输入包括两行，第一行是一个整数 n ($1 \leq n \leq 100000$)，表示果子的种类数。第二行包含 n 个整数，用空格分隔，第 i 个整数 a_i ($1 \leq a_i \leq 20000$) 是第 i 种果子的数目。

输出要求

输出包括一行，这一行只包含一个整数，也就是最小的体力耗费值。输入数据保证这个值小于231。

输入样例

3

1 2 9

输出样例

15

合并果子解题思路

- 贪心，容易想到每次合并最小的两堆。
- 先把 n 个数都push到小根堆里，合并 $n-1$ 次，每次取出堆顶的两个，更新答案，再把它们的和push到堆里。

priority_queue应用——丑数

题目描述

素因子都在集合{2, 3, 5, 7}的数称为
ugly number。

求第n大的丑数。 $n \leq 100000$ 。

输入样例

12

输出样例

15

丑数解题分析

- 先将2,3,5,7push进堆里，每次取出堆顶，将它分别乘上2,3,5,7push进堆里，弹出堆顶。第n次取的堆顶即为答案。

priority_queue应用—BlackBox

Black Box是一种原始的数据库。它可以储存一个整数数组，还有一个特别的变量*i*。最开始的时候**Black Box**是空的，而*i*等于0。这个**Black Box**要处理一串命令。命令只有两种：

ADD(x): 把 *x* 元素放进 **Black Box**; **GET**: *i* 加 1，然后输出 **Black box** 中第 *i* 小的数。

记住：第 *i* 小的数，就是 **Black Box**里的数的按从小到大的顺序排序后的第 *i* 个元素。

例如 我们来演示一下一个有11个命令的命令串。

序号	操作	<i>i</i>	数据库	输出
1	ADD(3)	0	3	
2	GET	1	3	3
3	ADD(1)	1	1,3	
4	GET	2	1,3	3
5	ADD(-4)	2	-4,1,3	
6	ADD(2)	2	-4,1,2,3	
7	ADD(8)	2	-4,1,2,3,8	
8	ADD(-1000)	2	-1000,-4,1,2,3,8	
9	GET	3	-1000,-4,1,2,3,8	1
10	GET	4	-1000,-4,1,2,3,8	2
11	ADD(2)	4	-1000,-4,1,2,2,3,8	

竞赛中常用STL算法

简单类

max:返回两个元素中较大一个。

min:返回两个元素中较小一个。

举例： `int a=1,b=2;`

`cout<<max(a,b)<<" "<<min(a,b)<<endl;`

2 1

swap: 交换存储在两个对象中的值

举例： `int x=10,y=20;`

`swap(x,y);`

`cout<<x<<" "<<y<<endl;`

20 10

区间类

sort:以升序重新排列指定范围内的元素。

举例： `int a[]={2,5,1,8,3,4,9,7,6};`

`sort(a,a+9);`

`for (int i = 0;i < 9;i++) cout<<a[i]<<" ";`

1 2 3 4 5 6 7 8 9

stable_sort: 以升序重新排列指定范围内的元素。不过保留相同元素间的顺序关系。即相同的元素在排序前出现在数组的前面，在排序后依然出现在数组的前面。

举例：`int a[]={1,5,1,8,3,1,3,7,8};`
`sort(a,a+9);`
`for (int i = 0;i < 9;i++) cout<<a[i]<<" ";`

1 1 1 3 3 5 7 8 8

reverse: 将指定范围内元素重新反序排序

举例: `vector<int> v1;`
`for ( int i = 0 ; i <= 9 ; i++ ) { v1.push_back( i ); }`
`reverse (v1.begin( ), v1.end( ) );`

执行后的v1: 9,8,7,6,5,4,3,2,1

举例: `int a[]={2,5,1,8,3,4,9,7,6};`
`reverse(a,a+9);`
`for (int i = 0;i < 9;i++) cout<<a[i]<<" ";`

6 7 9 4 3 8 1 5 2

random_shuffle: 对指定范围内的元素随机调整次序。

举例:

```
srand((unsigned int)time(0)); // 置换随机种子
```

```
vector<int> a;
```

```
for (int i=1;i<=9;i++) a.push_back(i);
```

```
random_shuffle(a.begin(),a.end());
```

举例: srand((unsigned int)time(0)); // 置换随机种子

```
int a[]={2,5,1,8,3,4,9,7,6};
```

```
random_shuffle(a,a+9);
```

```
for (int i = 0;i < 9;i++) cout<<a[i]<<" ";
```

第一次运行结果

3 7 1 8 9 4 5 2 6

第二次运行结果

7 5 4 1 6 2 8 3 9

第三次运行结果

4 2 8 3 6 9 1 7 5

next_permutation: 取出当前范围内的排列，并重新排序为下一个排列。

举例: `int main()`
`{`
 `int a[3];`
 `a[0]=1;a[1]=2;a[2]=3;`
 `do`
 `{`
 `cout<<a[0]<<" "<<a[1]<<"`
 `"<<a[2]<<endl;`
 `} while (next_permutation(a,a+3));`

1	2	3
1	3	2
2	1	3
2	3	1
3	1	2
3	2	1

举例：

5	6	7
5	7	6
6	5	7
6	7	5
7	5	6
7	6	5

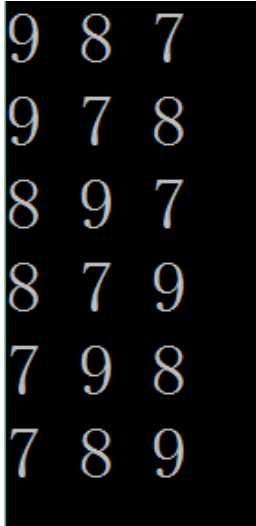
```
vector<int> a;  
a.push_back(5);  
a.push_back(6);  
a.push_back(7);  
vector<int>::iterator it  
do  
{  
for (it=a.begin(); it != a.end(); ++it)  
    cout << *it <<" ";  
    cout<<endl;  
} while(next_permutation(a.begin(),a.end()));
```

prev_permutation: 取出当前范围内的排列，并重新排序为上一个排列。

举例: `int main()`
{
 `int a[3];`
 `a[0]=3;a[1]=2;a[2]=1;`
 do
 {
 `cout<<a[0]<<" "<<a[1]<<"`
 `<<a[2]<<endl;`
 } while (`prev_permutation(a,a+3)`);

3	2	1
3	1	2
2	3	1
2	1	3
1	3	2
1	2	3

举例：



9	8	7
9	7	8
8	9	7
8	7	9
7	9	8
7	8	9

```
vector<int> a;  
a.push_back(9);  
a.push_back(8);  
a.push_back(7);  
vector<int>::iterator it;  
do  
{  
    for (it=a.begin(); it != a.end(); ++it)  
        cout << *it <<" ";  
    cout<<endl;  
} while  
(prev_permutation(a.begin(),a.end()));
```

merge: 将两个有序的序列合并为一个有序的序列。

举例：

```
int first[]={5,10,15,20,25};
int second[]={10,20,30,40,50};
vector<int> v(10); //需要事先留够空间
vector<int>::iterator it;
merge(first,first+5,second,second+5,v.begin());
for ( it=v.begin(); it != v.end(); ++it)
    cout << *it <<" ";
cout<<endl;
```

5 10 10 15 20 20 25 30 40 50

查找类

lower_bound(first,last,val): 在[first,last)区间进行二分查找，返回大于或等于val的第一个元素位置。

upper_bound(first,last,val): 在[first,last)区间进行二分查找，返回大于或等于val的最后一个元素位置。

举例：int a[]={0,1,2,2,3};
//注意，数组下标从0开始
int ans1=lower_bound(a,a+5,2)-a;
int ans2=upper_bound(a,a+5,2)-a;
cout<<ans1<<" "<<ans2<<endl;

2 4

find: 返回区间[first, end) 中第一个值等于value 的元素位置；若未找到，返回end。

举例：int a[]={2,5,1,8,3,4,9,7,6}; //注意，数组下标从0开始
int ans=find(a,a+9,8)-a;
cout<<ans<<endl;

3

举例：int a[]={2,5,1,8,3,4,9,7,6};
int ans=find(a,a+9,10)-a;
cout<<ans<<endl;

9

count: 统计某一值在一定范围内出现的次数

```
举例: vector<int>v;  
for(int n=-9;n<10;++n)  
    v.push_back(n);  
int num=count(v.begin(),v.end(),0);  
cout<<num<<endl;
```

1

```
举例: int a[]={2,2,2,2,2,2,2,2,2};  
int num=count(a,a+9,2);  
cout<<num<<endl;
```

9

其他

unique: 清除序列中重复元素，但是它不能真正删除元素。

举例：

```
int a[]={1,2,2,3,3,4,7,7,7,8,8};
```

```
unique(a,a+11);
```

```
for (int i=0;i<6;i++) //根据数据，我  
们知道，一共有6个不同的数字
```

```
cout<<a[i]<<" ";
```

1 2 3 4 7 8

但是如果循环为

```
for (int i=0;i<11;i++)
```

1 2 3 4 7 8 7 7 7 8 8

这也解释了为什么不是真正的删除元素

谢谢观看！

THE END